

A Project Report
on
**Testing and Validation of Estate Ease
Property Booking System**

*Submitted in partial fulfillment of the
requirement for the award of the degree of*

Master of Computer Applications (MCA)



Guru Gobind Singh Indraprastha University, Delhi

Guided By:

External Guide: Mr. Mo Masood Ansari

Internal Guide: Dr. Kriti Saroha

Submitted By:

Neha Gupta

(04411804424)



**Centre for Development of Advanced Computing,
Noida
2025 - 2026**

CERTIFICATE

I, **Neha Gupta**, Enroll No. **04411804424** certify that the Project Report/Dissertation (MCA-202) entitled “**Testing and Validation of Estate Ease Property Booking System**” is done by me and it is an authentic work carried out by me at **Qspiders, Noida**. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Student

Date:

Certified that the Project Report/Dissertation (MCA-202) entitled “**Testing and Validation of Estate Ease Property Booking System**” done by **Ms. Neha Gupta**, Enroll No. **04411804424**, is completed under my guidance.

Signature of the Guide

Date:

Name of the Guide: Mr. Mo Masood Ansari

Designation: Software Engineer

Address: Qspiders, Noida

ACKNOWLEDGEMENT

This work would not have been possible without the support of industry experts around me. Their judgement, constructive criticism and an eye for perfection have led this work towards completion. I would like to take the opportunity to acknowledge the valuable guidance rendered to me by Mr. Mo Masood Ansari (External Guide) and others at Qspiders. I am in a deep sense of admiration and their goodness and inexhaustible enthusiasm throughout my training tenure.

It is my proud privilege to express my gratitude to the entire management of CDAC, Noida and teachers of the institute for providing me with the opportunity to avail the excellent facilities along with knowledge and values that has proved to be of immense help at the very start of my career.

I wish to extend my sincere gratitude to Dr. Kriti Saroha, my faculty coordinator, for her guidance, encouragement and valuable suggestions.

I would also like to express my appreciation and admiration to all my friends and colleagues whose support and encouragement have been a constant source of assurance, guidance and strength to me which led to a wonderful experience during the tenure of this internship.

Neha Gupta
04411804424

TABLE OF CONTENTS

S No	Topic	Page No
1	Certificate	ii
2	Acknowledgement	iii
3	List of Figures	v
4	List of Tables	vi
5	List of Abbreviations	vi
6	Chapter 1 Introduction to the Project 1.1 About the Company 1.2 Introduction 1.3 Overview 1.4 Tools and Technology	7-11
7	Chapter 2: Systems Requirements Analysis 2.1 System Analysis 2.2 Feasibility Study 2.3 Hardware Requirement 2.4 Software Requirement	12-16
8	Chapter 3: System Design 3.1 System Architecture 3.2 Data Flow Diagram 3.3 Entity Relationship Diagram 3.4 API Design 3.5 Application Interface	17-25
9	Chapter 4: System Testing and Validation 4.1 Introduction to Testing 4.2 Functional Testing 4.3 Integration Testing 4.4 System Testing 4.5 API Testing using Postman 4.6 Postman Test Report	26-41
10	Chapter 5: System Implementation 5.1 System implementation 5.2 Deployment Requirements 5.3 User Roles and Access 5.4 Security and Authentication 5.5 System Outcome	42-43
11	Summary and Conclusions	44
12	References	45
13	Appendices	46-48

LIST OF FIGURES

S No	Figure Name	Page No
1	Fig 1: System Architecture	17
2	Fig 2: DFD	18
3	Fig 3: ER Diagram	19
4	Fig 4: API Design	20
5	Fig 5: Home Page	21
6	Fig 6: Login Page	21
7	Fig 7: Admin Dashboard	22
8	Fig 8: User Management Page	22
9	Fig 9: Booking Management Page	23
10	Fig 10: Enquiry Page	23
11	Fig 11: Add Property Page	24
12	Fig 12: Property Listing Page	24
13	Fig 13: Booking Schedule Page	25
14	Fig 14: Contact Page	25
15	Fig 15: adminLogin	34
16	Fig 16: userLogin	35
17	Fig 17: otpVerification	36
18	Fig 18: userBooking	37
19	Fig 19: getAllUsers	38
20	Fig 20: getAllBookings	39
21	Fig 21: updateBookingStatus	40
22	Fig 22: deletingUser	41

LIST OF TABLES

S No	Table Name	Page No
1	Table 1: Tools and Technology	11
2	Table 2: Functional Test Cases	28
3	Table 3: Integration Test Cases	30
4	Table 4: System Test Cases	32

LIST OF ABBREVIATIONS

1. SQL: Structured Query Language
2. API: Application Programming Interface
3. JSON: Java Script
4. CRUD: Create, Read, Update, Delete
5. HTTP: Hypertext Transfer Protocol

CHAPTER 1 - INTRODUCTION TO PROJECT

1.1 ABOUT THE COMPANY



QSpiders is a professional software training and development institute that provides industry-oriented training programs in software technologies, testing, and full-stack development. The organization focuses on enhancing technical skills and practical knowledge of students and professionals through hands-on project-based learning.

QSpiders offers training in various domains such as Java Development, Web Technologies, SQL, Manual Testing, Automation Testing, and Full Stack Development. The institute emphasizes practical implementation and real-time project exposure to prepare students for industry requirements.

The Noida branch of QSpiders provides a learning environment where students work on real-time projects and gain experience in software development methodologies, debugging, testing, and deployment processes. During the training period, various technologies including Java, Spring Boot, React.js, MySQL, and REST APIs were used for developing and testing web-based applications.

The project “Estate Ease” was developed as a team-based web application project during the training period. The system was designed to simplify property booking and management operations through an online platform. My primary contribution in the project involved testing and validation of the application modules and REST APIs using the Postman tool.

The training at QSpiders helped in improving technical knowledge, problem-solving abilities, API testing skills, and understanding of software development and testing workflows used in real-world applications.

1.2 INTRODUCTION

In today's digital era, online platforms have simplified many day-to-day activities by providing fast, secure, and convenient services. One such area is property searching and booking, where users often face difficulties in finding suitable properties, contacting owners, and managing bookings manually. Traditional property booking methods are time-consuming and may lead to communication issues, booking conflicts, and improper record management.

Estate Ease is a web-based property booking and management system developed to provide an efficient solution for online property searching and booking. The system allows users to view available properties, check property details, schedule visits, and send booking requests through an easy-to-use online platform. Administrators can manage property listings, booking requests, users, and customer enquiries through a centralized dashboard.

The application is developed using modern web technologies such as React.js for frontend development, Spring Boot for backend API development, and MySQL for database management. REST APIs are used for communication between frontend and backend modules.

The main focus of my contribution in this project was testing and validation of the application modules and backend APIs. Testing was performed to ensure that all modules work correctly and that data is properly transferred between frontend and backend systems. API testing was carried out using the Postman tool to verify request handling, response generation, status codes, and overall system functionality.

Different types of testing such as Functional Testing, Integration Testing, System Testing, and API Testing were performed during the project. The testing process helped in identifying errors, validating workflows, and ensuring smooth operation of the Estate Ease system for both users and administrators.

1.3 OVERVIEW

Estate Ease is a web-based property booking and management application designed to simplify the process of searching, viewing, and booking rental properties online. The system provides a centralized platform where users can explore available properties, schedule property visits, and send booking requests without the need for manual communication and paperwork.

The application consists of two main modules: the User Module and the Admin Module. The User Module allows users to register, log in, view property details, search properties based on location and price, and request bookings. The Admin Module enables administrators to manage property listings, monitor user activities, approve or reject booking requests, and handle customer enquiries.

The frontend of the application is developed using React.js, HTML, CSS, and Bootstrap to provide a responsive and user-friendly interface. The backend is developed using Java Spring Boot, which handles business logic and REST API services. MySQL is used as the database for storing user details, property information, booking records, and contact messages.

The project follows a three-tier architecture consisting of: Presentation Layer (Frontend), Business Logic Layer (Backend APIs), and Database Layer (MySQL Database).

The major contribution in this project was focused on software testing and API validation. Functional Testing was performed to verify the correct working of individual modules. Integration Testing ensured proper communication between frontend and backend components, while System Testing validated the complete working of the application as a whole.

API testing was carried out using the Postman tool. Different APIs related to user authentication, property management, booking management, and admin operations were tested using HTTP methods such as GET, POST, PUT, and DELETE. The testing process helped identify issues related to request handling, response validation, and data flow, ensuring that the application functions reliably and efficiently.

1.4 TOOLS AND TECHNOLOGY USED

The Estate Ease project was developed using various web technologies, frameworks, databases, and testing tools to ensure efficient system development and smooth application performance. The technologies used in the project supported frontend development, backend implementation, database management, and API testing.

The frontend of the application was developed using HTML, CSS, Bootstrap, JavaScript, and React.js. HTML and CSS were used to design and structure the web pages, while Bootstrap helped in creating a responsive and user-friendly interface. JavaScript was used for implementing interactive functionalities within the application. React.js was used for developing dynamic user interface components and handling frontend operations efficiently.

The backend of the application was developed using Java and Spring Boot. Java was used as the primary programming language for implementing the business logic of the system. Spring Boot framework was used for creating REST APIs and managing backend services. Spring Security was used to implement authentication and role-based authorization within the application.

MySQL was used as the database management system for storing user information, property details, booking records, and contact messages. MySQL Workbench was used for database creation and management purposes.

Postman was used for API testing and validation. Different REST APIs were tested using HTTP methods such as GET, POST, PUT, and DELETE to verify request handling, response generation, and proper communication between frontend and backend modules.

Visual Studio Code was used as the primary development environment during the project implementation process for both frontend and backend development.

The following table shows the major tools and technologies used in the project.

TOOL/TECHNOLOGY	PURPOSE
HTML	Structure of web pages
CSS	Styling and design
Bootstrap	Responsive user interface
JavaScript	Client-side scripting
React.js	Frontend development
Java	Backend development
Spring Boot	REST API development
Spring Security	Authentication and authorization
MySQL	Database management
Postman	API testing
VS Code	Development environment
MySQL Workbench	Database management tool

Table 1: Tools and Technology

CHAPTER 2 - SYSTEM REQUIREMENT ANALYSIS

2.1 SYSTEM ANALYSIS

System analysis is an important phase in software development that involves studying the existing system, identifying its limitations, and determining the requirements for the proposed system. The main objective of system analysis is to understand user needs and improve the overall efficiency of the system by providing a better and more reliable solution.

In the traditional property booking process, users generally depend on manual communication with brokers or property owners to search for available properties. Customers are often required to visit multiple locations physically to gather information related to property availability, pricing, and other details. This process is time-consuming and may result in communication delays, incorrect information, and booking conflicts.

Managing property records and booking requests manually also creates difficulties for administrators and property owners. Maintaining user records, property details, booking information, and customer enquiries without a centralized system increases the chances of data redundancy, poor record management, and operational inefficiency. As the number of users and properties increases, handling data manually becomes more difficult and less reliable.

Another major issue in the existing system is the lack of real-time booking management. Multiple users may request the same property at the same time, which can create booking conflicts and confusion. Manual tracking of booking status and visit schedules also affects the overall user experience and reduces system reliability.

The Estate Ease system was developed to overcome these limitations by providing a centralized web-based platform for property booking and management. The system allows users to search properties, view property details, schedule visits, and send booking requests through an online interface. Administrators can manage property listings, booking approvals, users, and customer enquiries efficiently through an admin dashboard.

The project also required proper testing and validation to ensure smooth functioning of the application modules and backend APIs. The major contribution in this project focused on testing activities including Functional Testing, Integration Testing, System Testing, and API Testing using Postman. These testing processes helped verify system functionality, validate API responses, and ensure proper communication between frontend and backend modules.

2.2 FEASIBILITY STUDY

A feasibility study is performed to evaluate whether the proposed system is practical, efficient, and beneficial for implementation. It helps in analysing different aspects of the system such as technical requirements, operational usability, and overall cost effectiveness. The feasibility study ensures that the proposed solution can be successfully developed and used within the available resources and constraints.

For the Estate Ease project, the feasibility study was conducted to determine whether the web-based property booking system could be implemented efficiently using the selected technologies and tools. The study mainly focused on technical feasibility, economic feasibility, and operational feasibility.

2.2.1 Technical Feasibility

Technical feasibility determines whether the required hardware, software, and technical resources are available for successful development and execution of the system. The Estate Ease project was developed using technologies such as React.js, Java, Spring Boot, MySQL, and Postman, which are widely used for modern web application development.

The system requirements for development and testing were easily available, and the selected technologies provided proper support for frontend development, backend API implementation, database management, and application testing. Postman was used for API testing and validation to ensure proper communication between frontend and backend modules. Therefore, the project was technically feasible and could be implemented effectively within the available technical environment.

2.2.2 Economic Feasibility

Economic feasibility evaluates the overall cost involved in the development and implementation of the system. The Estate Ease project was economically feasible because most of the technologies and tools used in the project were open-source and freely available.

The project did not require expensive hardware or licensed software for development and testing purposes. Technologies such as React.js, Spring Boot, MySQL, and Postman are available without additional cost, which reduced the overall development expenditure. Therefore, the project could be implemented within minimal cost while still providing efficient functionality and performance.

2.2.3 Operational Feasibility

Operational feasibility determines whether the proposed system can function properly within the intended environment and whether users can operate the system effectively. The Estate Ease application was designed with a simple and user-friendly interface to ensure ease of use for both users and administrators.

The system automates property searching, booking management, and administrative operations, reducing manual effort and improving overall efficiency. Proper testing and validation were also performed to ensure reliable system performance and smooth user interaction. As the application fulfils user requirements and simplifies property management operations, the system is operationally feasible.

2.3 HARDWARE REQUIREMENTS

The hardware requirements for the Estate Ease project were minimal and sufficient for web application development, database management, and API testing activities. The system was developed and tested on a standard computer system capable of supporting frontend development, backend processing, and database operations efficiently.

The hardware configuration used during the development and testing of the project is listed below:

Hardware Component	Specification
Processor	Intel Core i5 or above
RAM	8 GB
Storage	256 GB SSD or above
System Type	64-bit Operating System
Input Devices	Keyboard and Mouse
Output Device	Monitor
Internet Connection	Required for development and testing

The available hardware resources were sufficient for running development tools, database services, frontend applications, backend servers, and API testing tools simultaneously without significant performance issues.

2.4 SOFTWARE REQUIREMENTS

Various software technologies and development tools were used for the implementation and testing of the Estate Ease project. These software requirements supported frontend development, backend API creation, database management, and application testing processes.

The software requirements used during the development and testing of the project are listed below:

Software / Tool	Purpose
Operating System	Windows 10 or above
HTML	Structure of web pages
CSS	Styling and user interface design
Bootstrap	Responsive frontend design
JavaScript	Client-side scripting
React.js	Frontend development
Java	Backend programming
Spring Boot	REST API development
Spring Security	Authentication and authorization
MySQL	Database management
MySQL Workbench	Database administration
Postman	API testing and validation
Visual Studio Code	Development environment
Web Browser	Chrome or Edge

These software tools and technologies were selected because they provide efficient support for full-stack web application development and testing. Postman was mainly used for testing REST APIs and validating communication between frontend and backend-modules.

CHAPTER 3 - SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

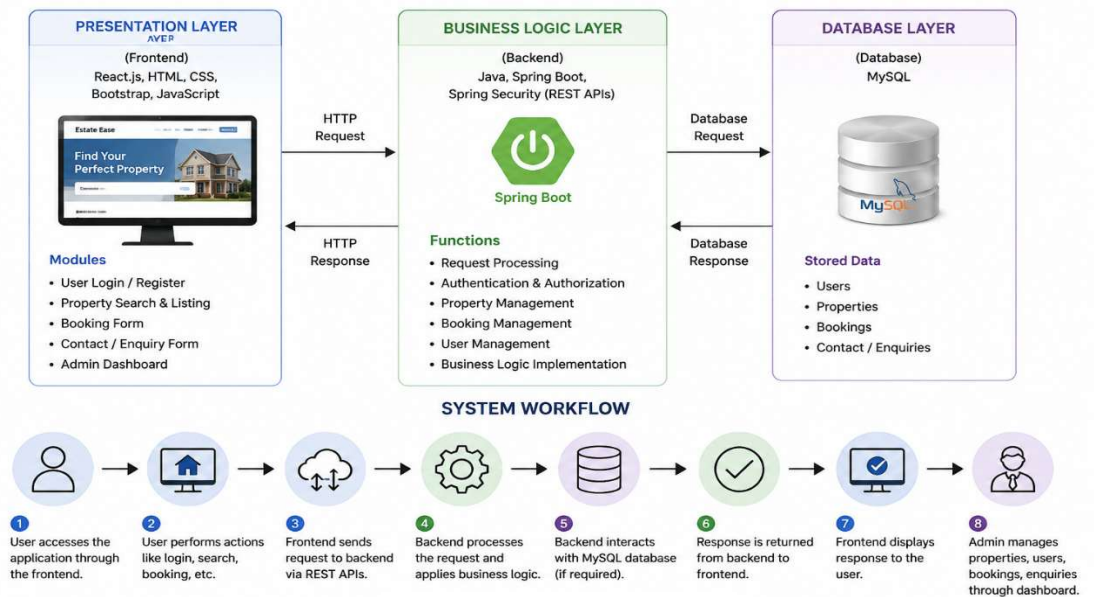


Fig 1: System Architecture

The Estate Ease application follows a three-tier architecture consisting of the Presentation Layer, Business Logic Layer, and Database Layer. This architecture helps in maintaining proper separation between frontend, backend, and database operations, making the system organized and easier to manage.

The **Presentation Layer** is the frontend of the application developed using React.js, HTML, CSS, Bootstrap, and JavaScript. This layer provides the user interface for property searching, booking requests, user login, and admin operations.

The **Business Logic Layer** is developed using Java and Spring Boot. It handles application logic, request processing, authentication, booking management, and communication between the frontend and database through REST APIs.

The **Database Layer** uses MySQL for storing user details, property information, booking records, and contact messages.

The workflow of the system starts when a user sends a request from the frontend interface. The request is processed by the backend APIs, which interact with the database whenever required. The response is then returned to the frontend and displayed to the user.

3.2 DATA FLOW DIAGRAM (DFD)

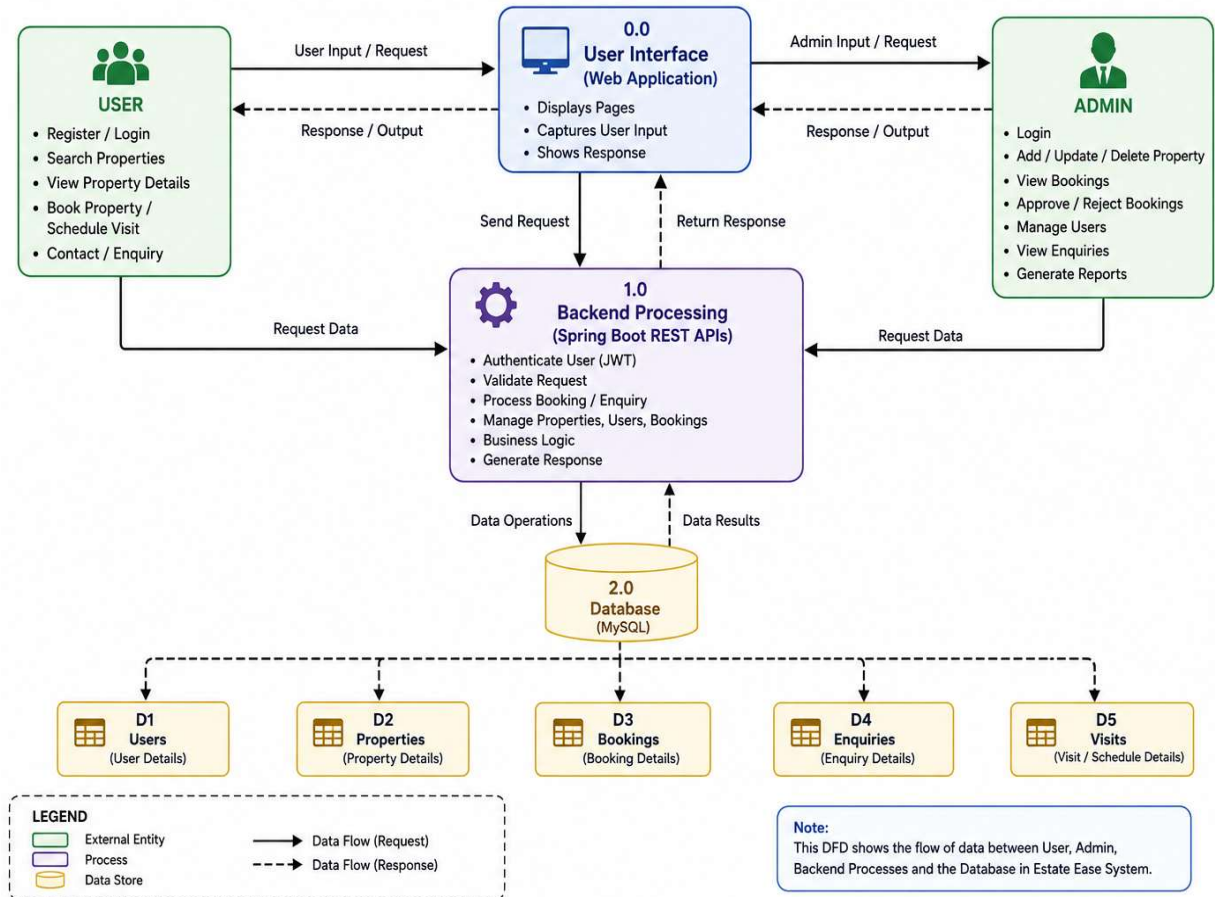


Fig 2: DFD

3.3 ENTITY RELATIONSHIP (ER) DIAGRAM

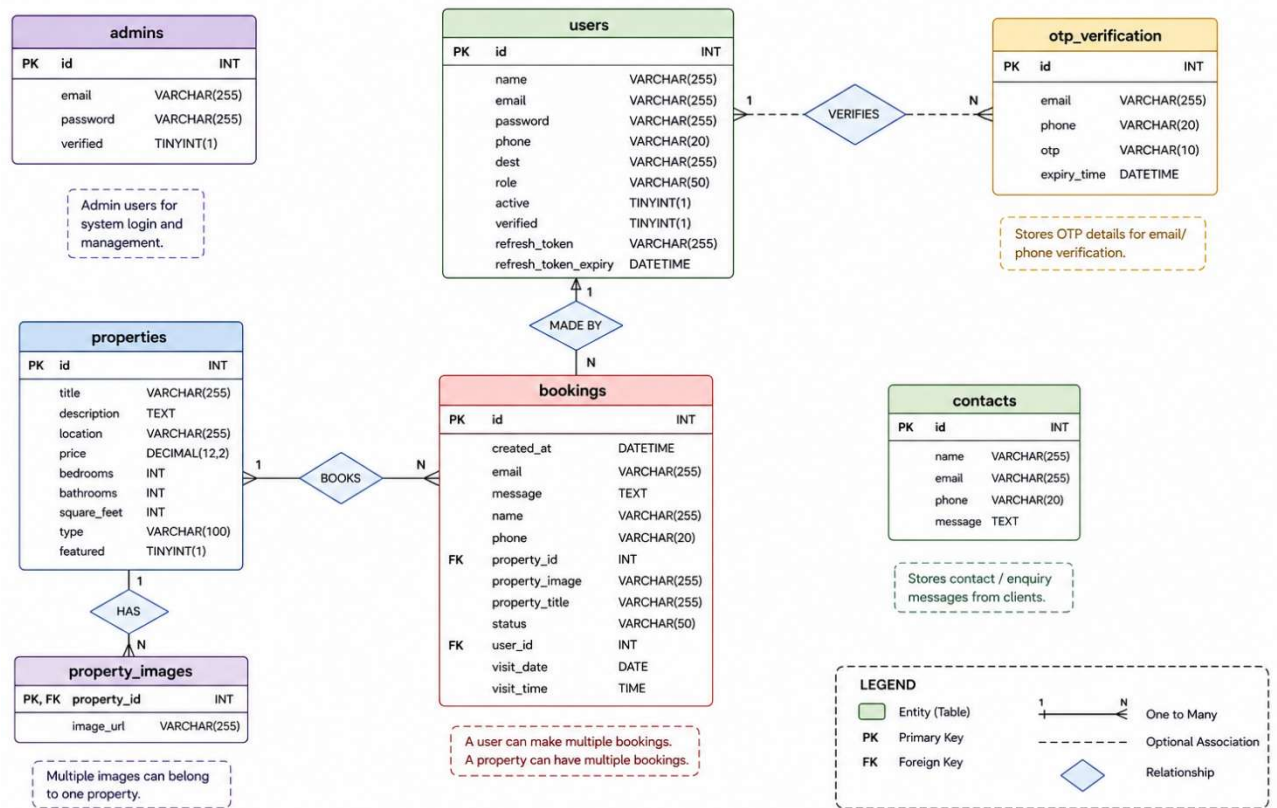


Fig 3: ER Diagram

3.4 API DESIGN

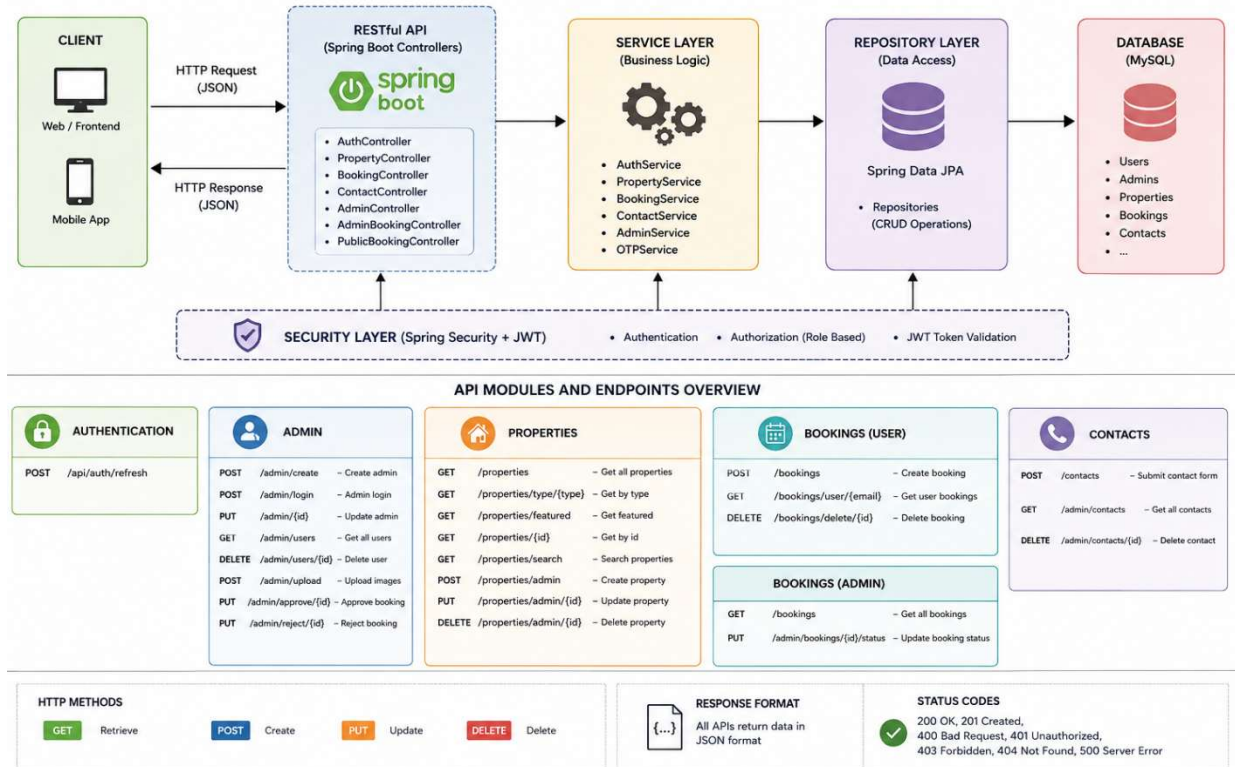


Fig 4: API Design

The system uses RESTful APIs to enable communication between the frontend and backend of the application. The APIs were developed using Spring Boot and use standard HTTP methods such as GET, POST, PUT, and DELETE for performing different operations. The API layer handles major functionalities of the system including user authentication, property management, booking management, contact handling, and administrative operations. Data is exchanged in JSON format between the client and server.

The APIs follow a layered architecture consisting of controller, service, and repository layers. The controller layer manages HTTP requests, the service layer handles business logic, and the repository layer performs database operations.

JWT-based authentication and role-based authorization were implemented using Spring Security to secure protected endpoints. Administrative APIs are restricted to authorized users only. The APIs were tested using Postman to verify request handling, response generation, and CRUD operations. Overall, the API design provides a secure and efficient communication mechanism for the system.

3.5 APPLICATION INTERFACE

The Estate Ease application provides a simple and user-friendly interface for both users and administrators. Different interfaces were developed for property browsing, booking management, user authentication, and administrative operations. The following figures show the major interfaces of the application.

3.5.1 Home Page

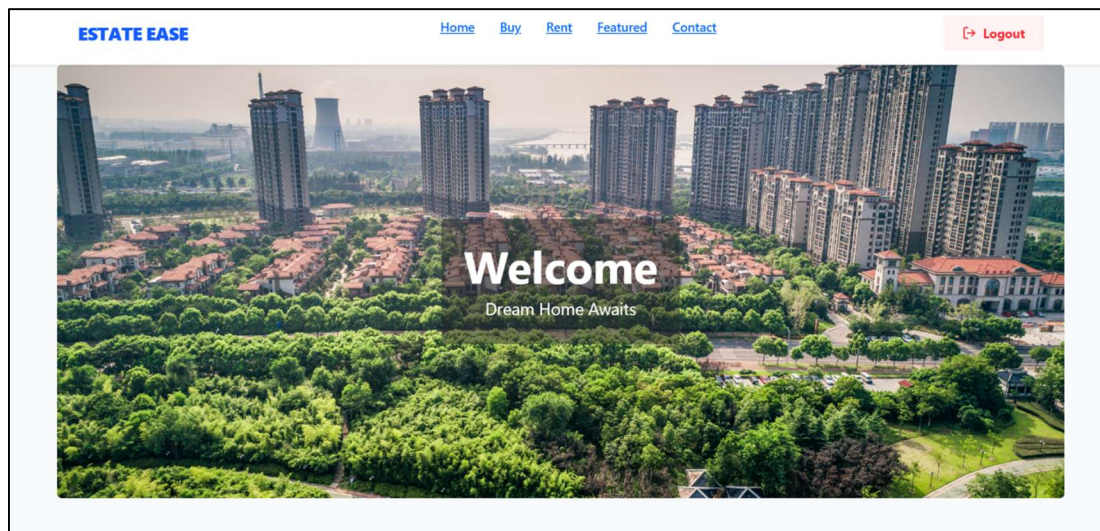


Fig 5: Home Page

The home page provides the main interface of the Estate Ease application. It displays featured properties, navigation menus, and access to different modules of the system.

3.5.2 User Login Page

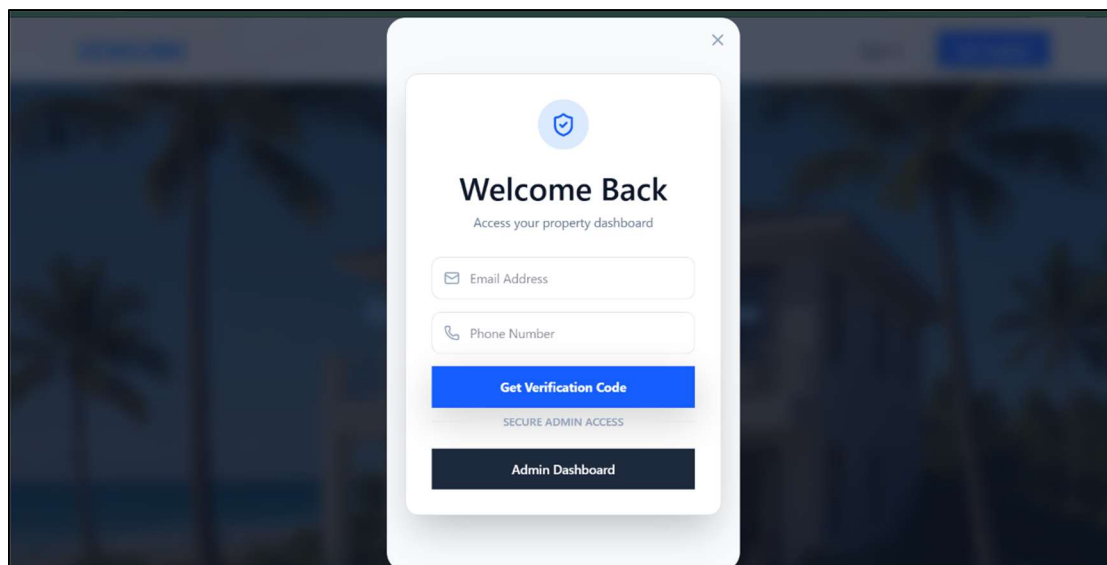


Fig 6: Login Page

The login page allows registered users and administrators to securely access the application using valid credentials.

3.5.3 Admin Dashboard

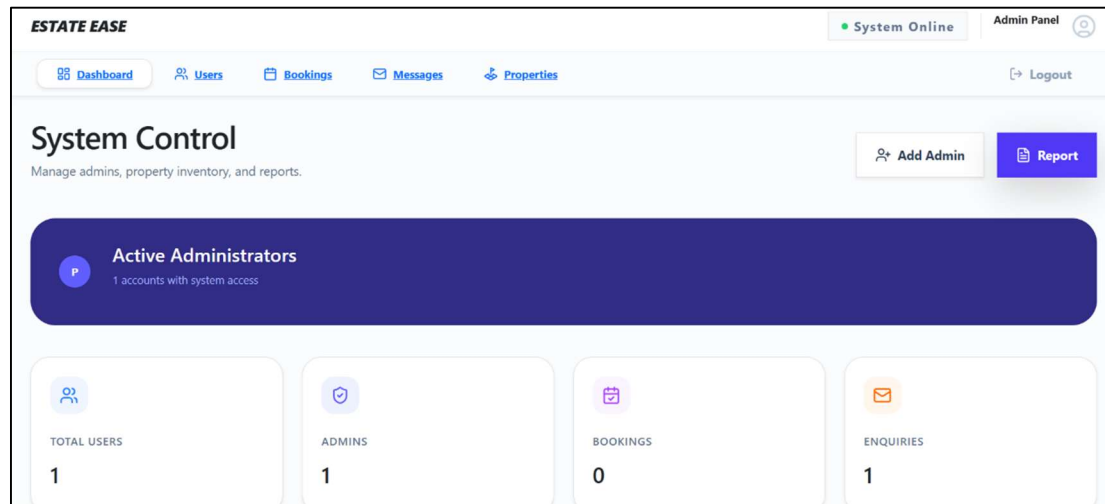


Fig 7: Admin Dashboard

The admin dashboard provides administrative functionalities such as user management, booking management, and property management.

3.5.4 User Management Page

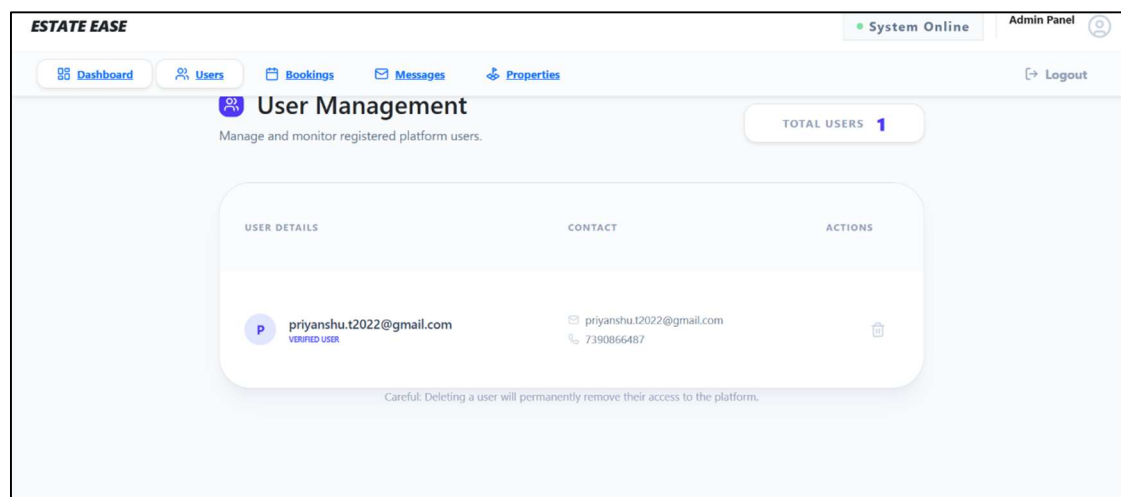


Fig 8: User Management Page

This interface allows administrators to view and manage registered users within the application.

3.5.5 Booking Management Page

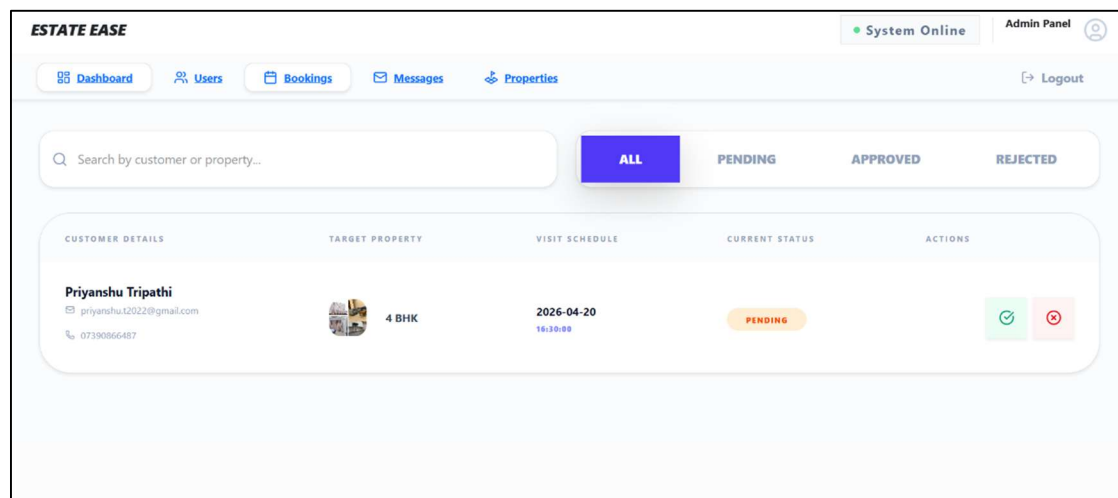


Fig 9: Booking Management

The booking management page allows administrators to view booking requests and update booking status.

3.5.6 Customer Enquiry Page

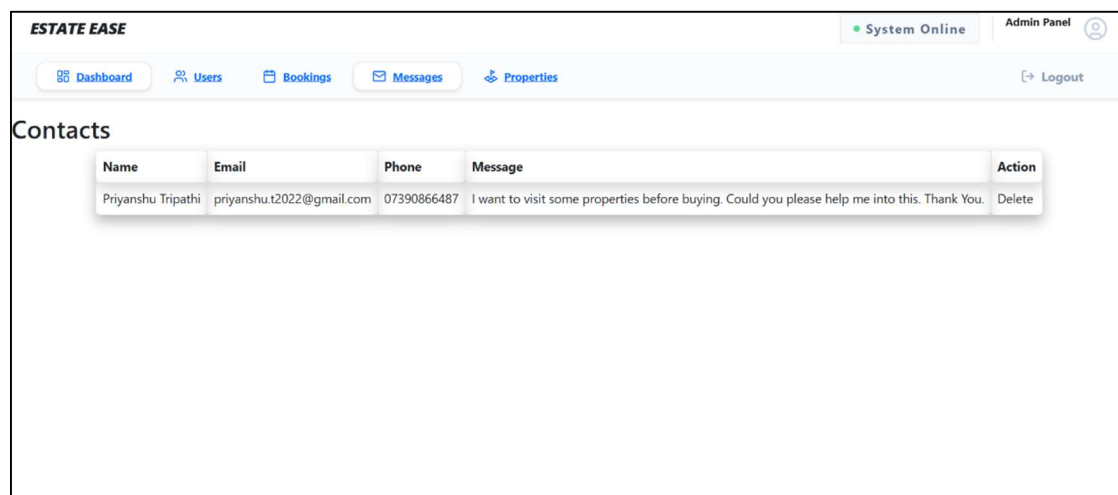


Fig 10: Enquiry Page

This page allows administrators to view customer enquiries and messages submitted through the contact form.

3.5.7 Add Property Page

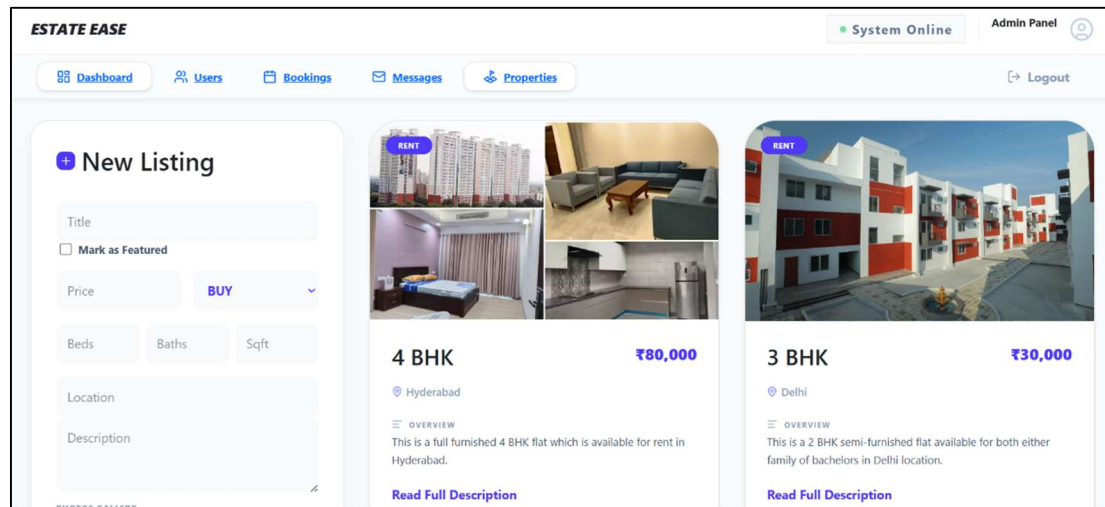


Fig 11: Add Property Page

The add property page allows administrators to add new property listings along with property details and images.

3.5.8 Property Listing Page

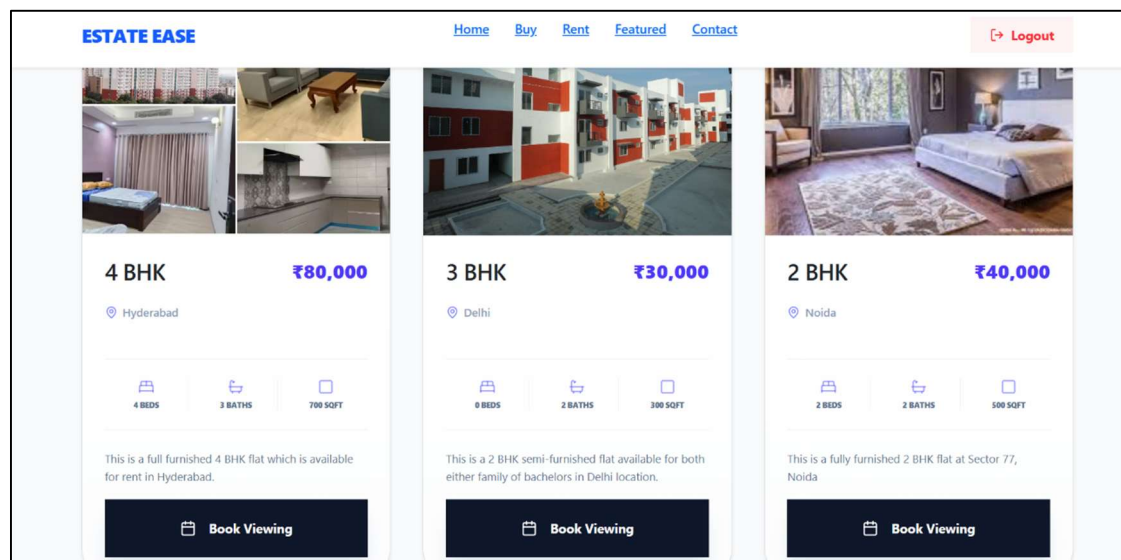


Fig 12: Property Listing Page

The property listing page allows users to view available properties along with details such as location, price, property type, and property images.

3.5.9 Booking Schedule Page

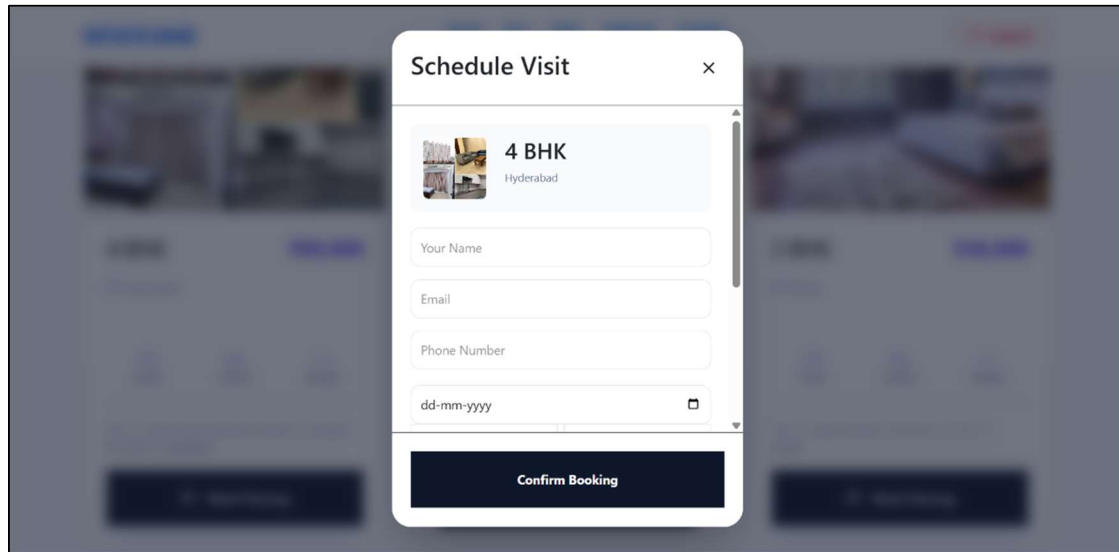
A screenshot of a mobile application showing a 'Schedule Visit' modal. The modal has a title bar with a close button. Below the title, there is a property card for '4 BHK' in 'Hyderabad' with a small image. Underneath the card are four input fields: 'Your Name', 'Email', 'Phone Number', and a date field with a calendar icon and the placeholder 'dd-mm-yyyy'. At the bottom of the modal is a dark blue button labeled 'Confirm Booking'. The background is a blurred view of a property listing page.

Fig 13: Booking Schedule Page

This interface allows users to schedule property visits and submit booking requests by entering the required details.

3.5.10 Contact Page

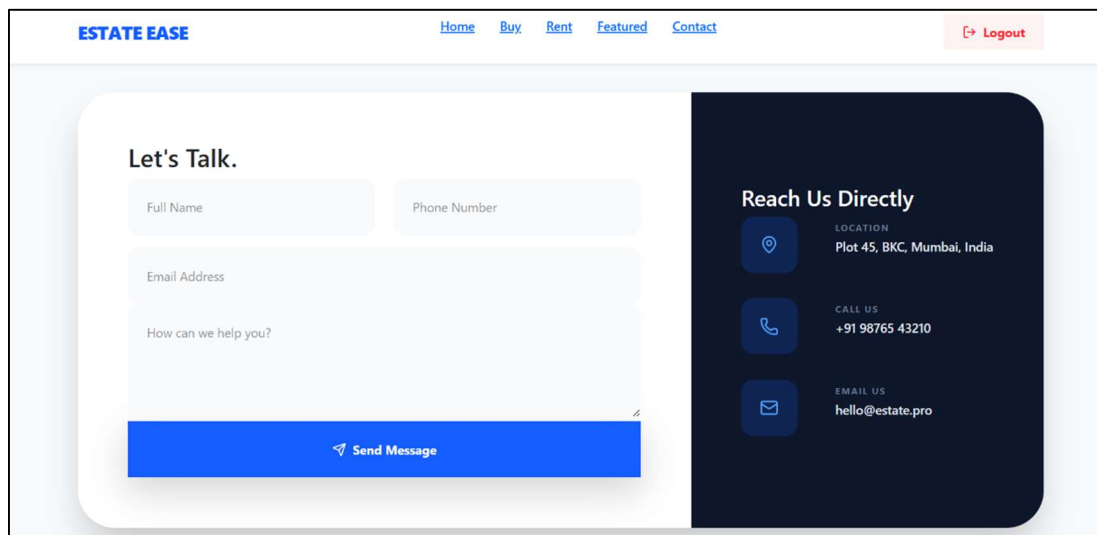
A screenshot of a web application's 'Contact' page. The page has a header with the 'ESTATE EASE' logo and navigation links: 'Home', 'Buy', 'Rent', 'Featured', and 'Contact'. A 'Logout' button is in the top right. The main content area is divided into two sections. The left section, titled 'Let's Talk.', contains a contact form with fields for 'Full Name', 'Phone Number', 'Email Address', and a text area for 'How can we help you?'. A blue 'Send Message' button is at the bottom of the form. The right section, titled 'Reach Us Directly', provides direct contact information: 'LOCATION: Plot 45, BKC, Mumbai, India', 'CALL US: +91 98765 43210', and 'EMAIL US: hello@estate.pro', each with a corresponding icon.

Fig 14: Contact Page

The contact page allows users to send enquiries and communicate with administrators regarding property-related information.

CHAPTER 4 - SYSTEM TESTING & VALIDATION

4.1 INTRODUCTION TO TESTING

Software testing is an important phase in the software development life cycle that helps in verifying whether the application functions correctly according to the specified requirements. Testing is performed to identify errors, validate system functionality, and ensure reliable communication between different modules of the application. Proper testing improves software quality, system reliability, and overall user experience.

In the Estate Ease project, testing was performed to validate the working of different modules such as user authentication, property management, booking operations, contact management, and administrative functionalities. The testing process helped ensure that the application performs accurately and efficiently for both users and administrators.

Different types of testing were carried out during the project including Functional Testing, Integration Testing, System Testing, and API Testing using Postman. Functional testing was used to verify the correct working of individual modules and features of the application. Integration testing was performed to ensure smooth communication between frontend and backend modules. System testing validated the complete workflow and overall behaviour of the application.

API testing was one of the major parts of the testing process. REST APIs developed using Spring Boot were tested using the Postman tool to verify request handling, response generation, status codes, and backend functionality. Different API requests and responses were validated to ensure proper communication between frontend interfaces and backend services.

The testing process helped identify and resolve issues related to request handling, data validation, authentication, booking workflows, and API responses. Proper testing ensured that the Estate Ease application operates efficiently, securely, and reliably.

4.2 FUNCTIONAL TESTING

Functional testing was performed to verify whether the individual modules and functionalities of the Estate Ease application were working according to the specified requirements. The main objective of functional testing was to ensure that each feature of the system produced the expected output for valid user actions and handled invalid operations properly.

Different modules of the application were tested individually during the functional testing process. The login and registration module was tested to verify user authentication, credential validation, and successful access to the system. The property management module was tested to ensure proper display of property details, featured properties, property images, and property information.

The booking module was tested to verify booking request submission, visit scheduling, booking status updates, and prevention of invalid booking operations. The contact module was tested to validate message submission and storage of customer enquiry details.

Administrative functionalities such as property management, booking approval, user management, and enquiry handling were also tested to ensure correct operation through the admin dashboard.

Functional testing helped identify issues related to form validation, incorrect input handling, response generation, and module functionality. After resolving the identified issues, the modules operated correctly according to the expected system behaviour.

The following table shows some functional test cases performed during the testing process.

Test Case ID	Module	Test Scenario	Expected Result	Status
TC01	User Registration	Register user with valid details	User account should be created successfully	Pass
TC02	User Registration	Register user with invalid email or empty fields	Validation error message should be displayed	Pass
TC03	Login Module	Login with valid credentials	User should login successfully	Pass
TC04	Login Module	Login with invalid password	Error message should be displayed	Pass
TC05	Property Module	View available property listings	Properties should display correctly	Pass
TC06	Property Module	Search properties by location	Matching properties should be displayed	Pass
TC07	Booking Module	Submit booking request with valid details	Booking request should be saved successfully	Pass
TC08	Booking Module	Attempt duplicate booking for same property slot	System should prevent duplicate booking	Pass
TC09	Contact Module	Submit customer enquiry through contact form	Contact message should be stored successfully	Pass
TC10	Admin Module	Login using valid admin credentials	Admin dashboard should open successfully	Pass
TC11	Admin Module	Approve booking request	Booking status should update successfully	Pass
TC12	OTP Verification Module	Verify OTP using valid OTP code	User verification should complete successfully	Pass

Table 2: Functional Test Cases

4.3 INTEGRATION TESTING

Integration testing was performed to verify proper communication and data flow between different modules of the Estate Ease application. The main objective of integration testing was to ensure that the frontend, backend APIs, and database modules interact correctly and exchange data without errors.

During the testing process, different integrated workflows of the application were validated. The frontend developed using React.js was connected with backend REST APIs developed using Spring Boot, while MySQL was used for storing and retrieving application data. Integration testing helped verify that requests sent from the frontend were processed correctly by the backend and appropriate responses were returned successfully.

The login module was tested to ensure that user credentials entered on the frontend were validated correctly through backend APIs and authenticated user data was fetched from the database. The property module was tested to verify successful retrieval and display of property information from the database through APIs.

The booking module was tested to ensure that booking requests submitted by users were properly processed, stored in the database, and reflected correctly in the admin dashboard. Integration testing was also performed for contact form submissions and OTP verification functionalities to validate smooth communication between application layers.

The testing process helped identify issues related to API integration, request handling, data transfer, and database connectivity. After resolving the identified issues, all integrated modules worked correctly and ensured smooth operation of the application.

The following table shows some integration test cases performed during the testing process.

Test Case ID	Integrated Modules	Test Scenario	Expected Result	Status
IT01	Frontend + Login API + Database	User logs in using valid credentials	User should login successfully and dashboard should open	Pass
IT02	Frontend + Registration API + Database	User submits registration form	User data should be stored successfully in database	Pass
IT03	Frontend + Property API + Database	Fetch and display property listings	Property details should display correctly on frontend	Pass
IT04	Frontend + Booking API + Database	User submits booking request	Booking details should be stored successfully in database	Pass
IT05	Booking Module + Admin Dashboard	Admin views booking requests	Submitted bookings should display correctly in admin panel	Pass
IT06	Admin Module + Booking API + Database	Admin approves booking request	Booking status should update successfully in database	Pass
IT07	Frontend + Contact API + Database	User submits contact enquiry	Contact message should store correctly in database	Pass
IT08	Frontend + OTP API + Database	User verifies OTP during authentication	OTP verification should complete successfully	Pass

Table 3: Integration Test Cases

4.4 SYSTEM TESTING

System testing was performed to validate the complete functionality and overall behaviour of the Estate Ease application. The main objective of system testing was to ensure that all modules of the application work together correctly as a complete integrated system and satisfy the specified requirements.

During system testing, the complete workflows of the application were tested from the user interface to backend processing and database operations. Different user activities such as registration, login, property searching, booking requests, contact form submission, and admin operations were executed to verify smooth system functionality.

The testing process ensured that the frontend interface, backend APIs, and database components interacted properly without errors. End-to-end workflows were validated to confirm that the application handled user requests correctly and generated appropriate responses.

System testing also helped verify application reliability, data consistency, navigation flow, and booking management operations. The complete booking workflow, including booking request submission and admin approval, was tested successfully to ensure proper system behaviour.

The following table shows some system test cases performed during the testing process.

Test Case ID	System Workflow	Expected Result	Status
ST01	User Registration → Login	User should register and login successfully	Pass
ST02	Login → Search Property → View Property Details	Property details should display correctly	Pass
ST03	Login → Schedule Property Visit → Submit Booking Request	Booking request should be submitted successfully	Pass
ST04	User Submits Contact Form	Contact enquiry should be stored successfully	Pass
ST05	Admin Login → View Booking Requests	Booking requests should display correctly in admin dashboard	Pass
ST06	Admin Approves Booking Request	Booking status should update successfully	Pass
ST07	Admin Adds New Property → Property Displays on Frontend	Newly added property should display correctly	Pass
ST08	Complete Workflow: Login → Search Property → Book Visit → Admin Approval	Entire system workflow should execute successfully without errors	Pass

Table 4: System Test Cases

4.5 API TESTING USING POSTMAN

API testing was performed using the Postman tool to validate the REST APIs developed for the Estate Ease application. The purpose of API testing was to verify proper communication between frontend and backend modules and ensure that the APIs handled requests and responses correctly.

Different HTTP methods such as GET, POST, PUT, and DELETE were used to test various API endpoints. The APIs were validated using request parameters, response data, and status codes.

The testing process included APIs related to:

- User authentication
- Property management
- Booking operations
- Contact enquiries
- OTP verification

API testing helped verify request handling, response generation, data validation, and backend functionality. It also helped identify issues related to API integration and response handling during the testing process.

The following sections show the API requests, responses, and validation results tested using Postman.

4.5.1 Admin Login

The Admin Login API was tested using Postman to verify secure authentication and successful access to the admin dashboard. The API validates admin credentials and allows authorized administrators to access administrative functionalities of the Estate Ease application.

The API was tested using valid and invalid login credentials to verify request handling, response generation, and authentication validation. Proper status codes and responses were received during the testing process.

4.5.2 USER LOGIN API TESTING

The User Login API was tested using Postman to verify OTP-based user authentication in the Estate Ease application. The API validates user details and sends a One-Time Password (OTP) to the registered email address for verification.

API Endpoint	HTTP Method	Purpose
/api/auth/send-otp	POST	Send OTP for user authentication

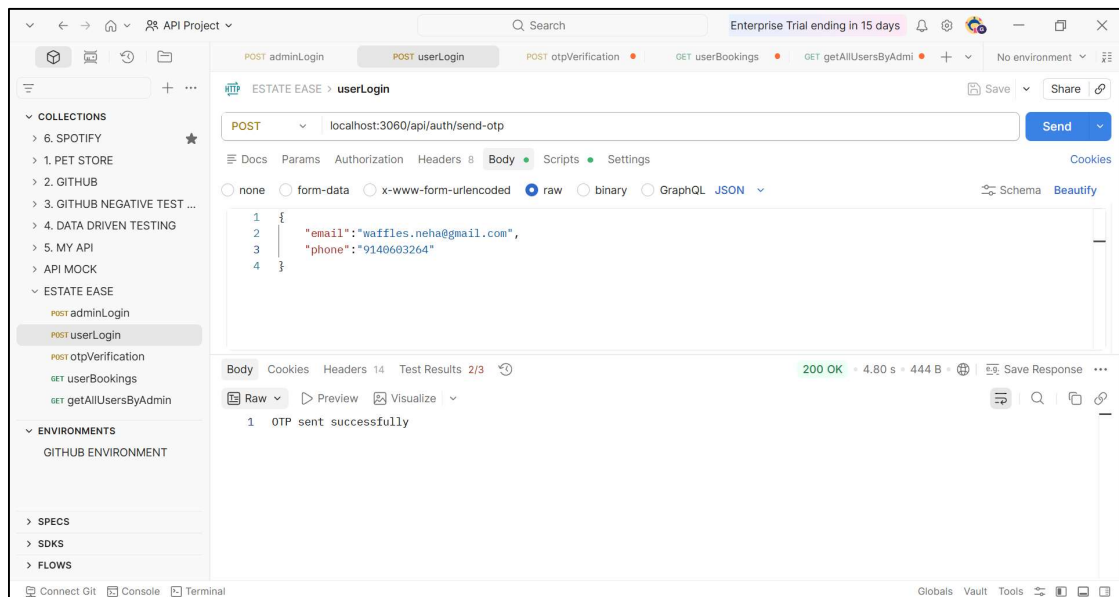


Fig 16: userLogin

Request Body

```
{
  "email": "waffles.neha@gmail.com",
  "phone": "9140603264"
}
```

Response Body

OTP sent successfully

Response Validation

The API returned status code **200 OK** and successfully generated the OTP for user authentication. This verified proper request handling and OTP generation functionality.

4.5.3 OTP VERIFICATION API TESTING

The OTP Verification API was tested to validate successful OTP authentication and token generation for users. The API verifies the OTP entered by the user and provides authentication tokens for secure system access.

API Endpoint	HTTP Method	Purpose
/api/auth/verify-otp	POST	Verify user OTP

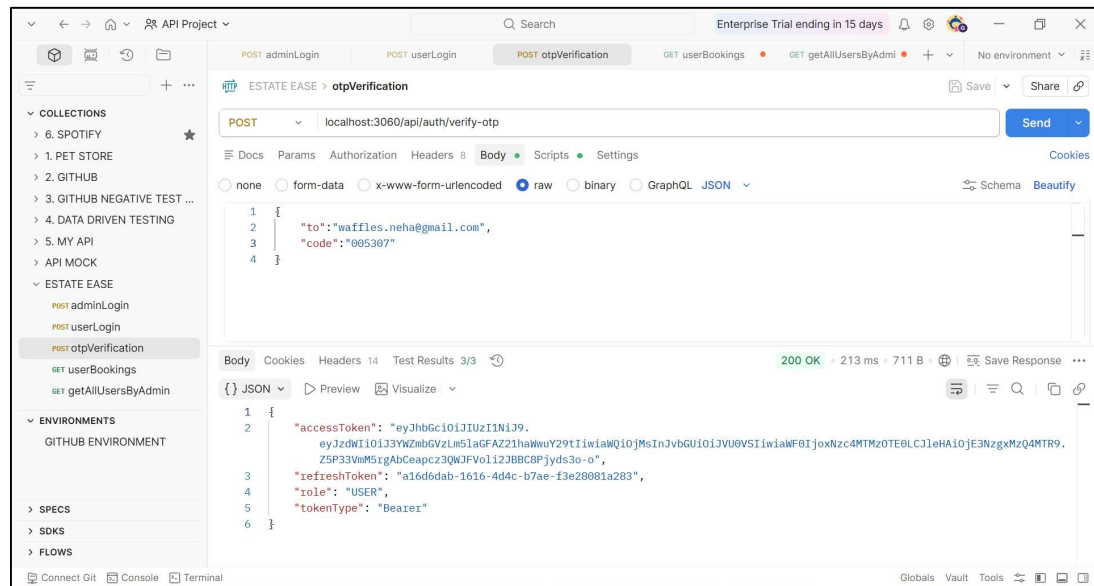


Fig 17: otpVerification

Request Body

```
{
  "to": "waffles.neha@gmail.com",
  "code": "005307"
}
```

Response Body

```
{
  "accessToken": "Generated Access Token",
  "refreshToken": "Generated Refresh Token",
  "role": "USER",
  "tokenType": "Bearer"
}
```

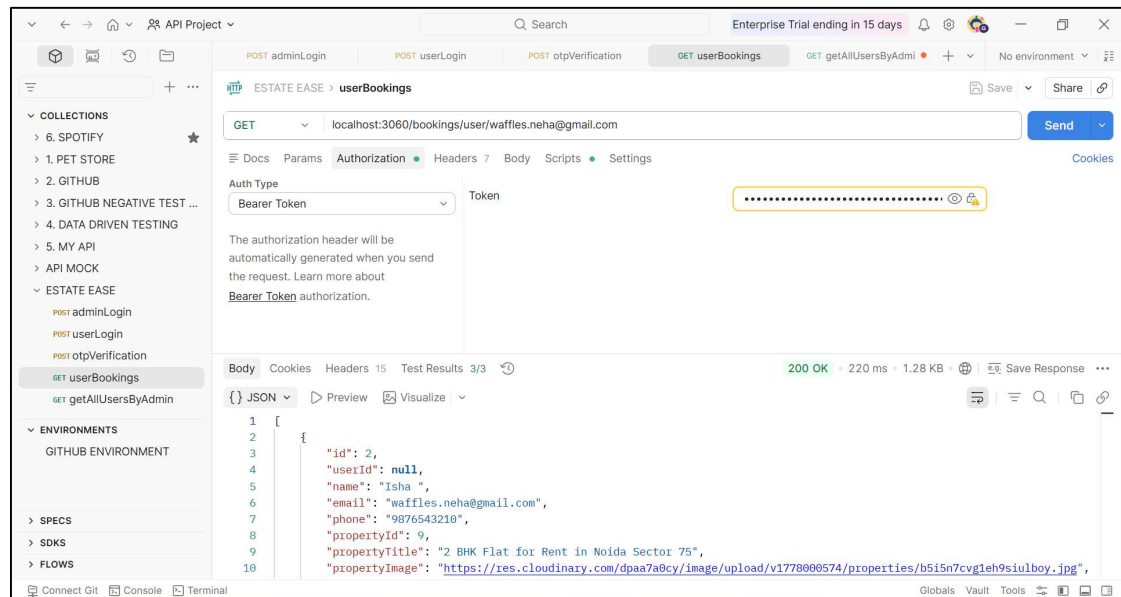
Response Validation

The API returned status code **200 OK** and generated authentication tokens after successful OTP verification. The response confirmed valid user authentication and secure access to the application.

4.5.4 USER BOOKING API TESTING

The User Booking API was tested using Postman to verify successful retrieval of booking requests made by users in the Estate Ease application. The API fetches booking details associated with a specific user and returns the data in JSON format.

API Endpoint	HTTP Method	Purpose
/bookings/user/{email}	GET	Fetch booking requests of a user



Request Details Fig 18: userBooking

The API request was sent using the GET method with the user email included in the endpoint URL. Bearer Token authorization was used to access the protected API.

Response Validation

The API returned status code **200 OK** and successfully fetched the booking details associated with the specified user. The response contained booking information such as user details, property details, property image, and booking status in JSON format.

Sample Response Data

```
{
  "id": 2,
  "name": "Isha",
  "email": "waffles.neha@gmail.com",
  "phone": "9876543210",
  "propertyTitle": "2 BHK Flat for Rent in Noida Sector 75"
}
```

4.5.5 FETCH ALL USERS API TESTING

The Fetch All Users API was tested using Postman to verify successful retrieval of all registered users by the administrator. The API allows the admin to access user information stored in the database.

API Endpoint	HTTP Method	Purpose
/admin/users	GET	Fetch all registered users

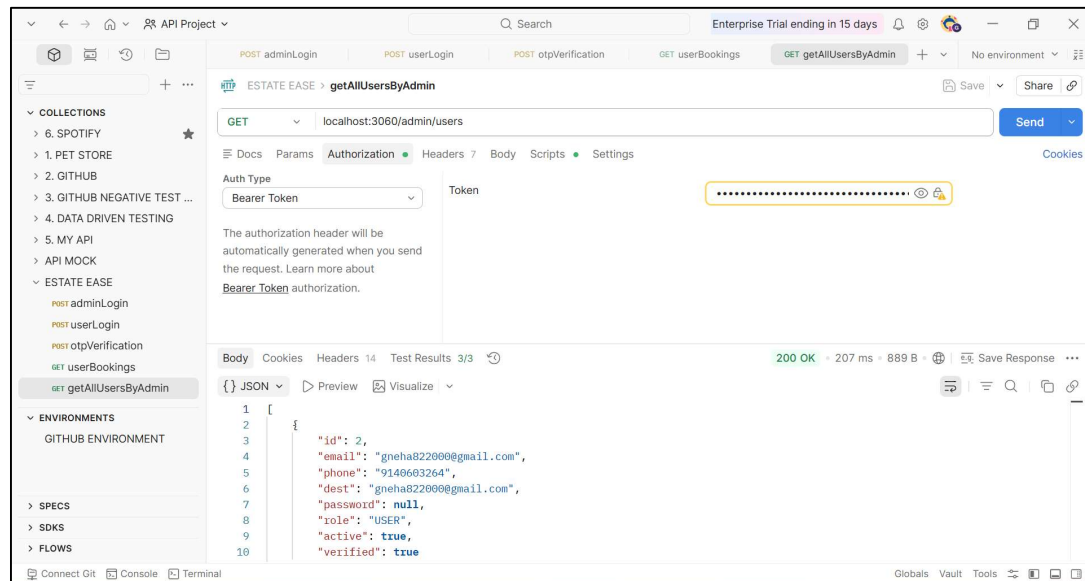


Fig 19: getAllUsers

Request Details

The API request was sent using the GET method with Bearer Token authorization to provide secure admin access.

Response Validation

The API returned status code **200 OK** and successfully fetched all registered user details from the database. The response contained user information such as email, phone number, role, active status, and verification status in JSON format.

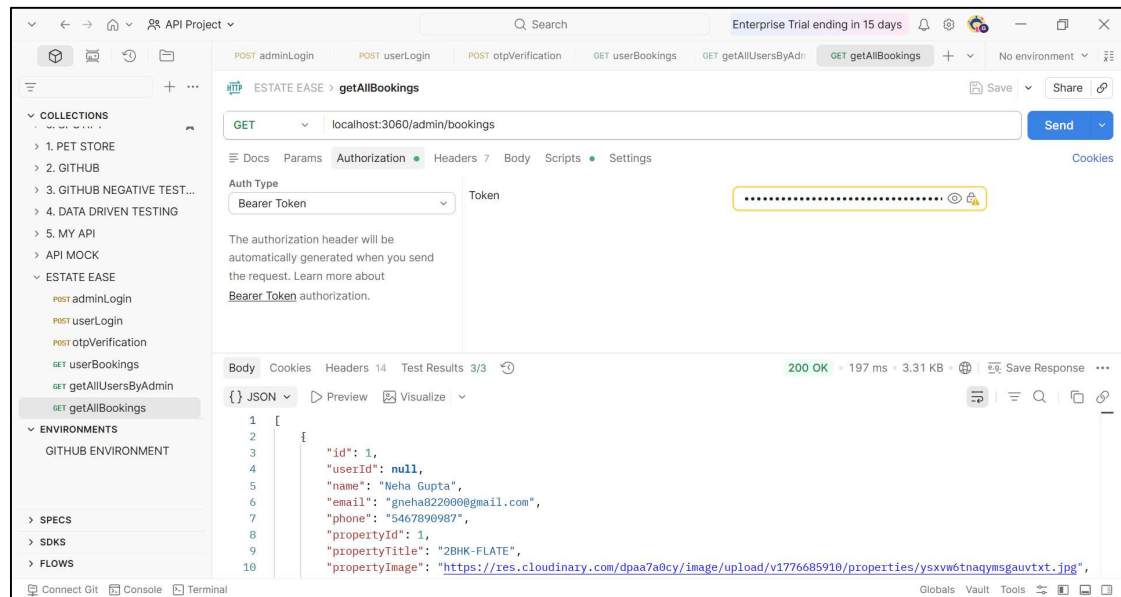
Sample Response Data

```
{
  "id": 2,
  "email": "gneha822000@gmail.com",
  "phone": "9140603264",
  "role": "USER",
  "active": true,
  "verified": true
}
```

4.5.6 FETCH ALL BOOKINGS API TESTING

The Fetch All Bookings API was tested using Postman to verify successful retrieval of all booking requests by the administrator. The API allows the admin to access booking information submitted by users in the Estate Ease application.

API Endpoint	HTTP Method	Purpose
/admin/bookings	GET	Fetch all booking requests



Request Details

Fig 20: getAllBookings

The API request was sent using the GET method with Bearer Token authorization to provide secure admin access.

Response Validation

The API returned status code **200 OK** and successfully fetched all booking request details from the database. The response contained customer details, property details, booking information, and property images in JSON format.

Sample Response Data

```
{
  "id": 1,
  "name": "Neha Gupta",
  "email": "gneha822000@gmail.com",
  "phone": "5467890987",
  "propertyTitle": "2BHK-FLATE"
}
```

4.5.7 UPDATE BOOKING STATUS API TESTING

The Update Booking Status API was tested using Postman to verify successful modification of booking status by the administrator. The API allows the admin to approve or reject booking requests submitted by users.

API Endpoint	HTTP Method	Purpose
/admin/bookings/{id}/status?status=APPROVED	PUT	Update booking request status

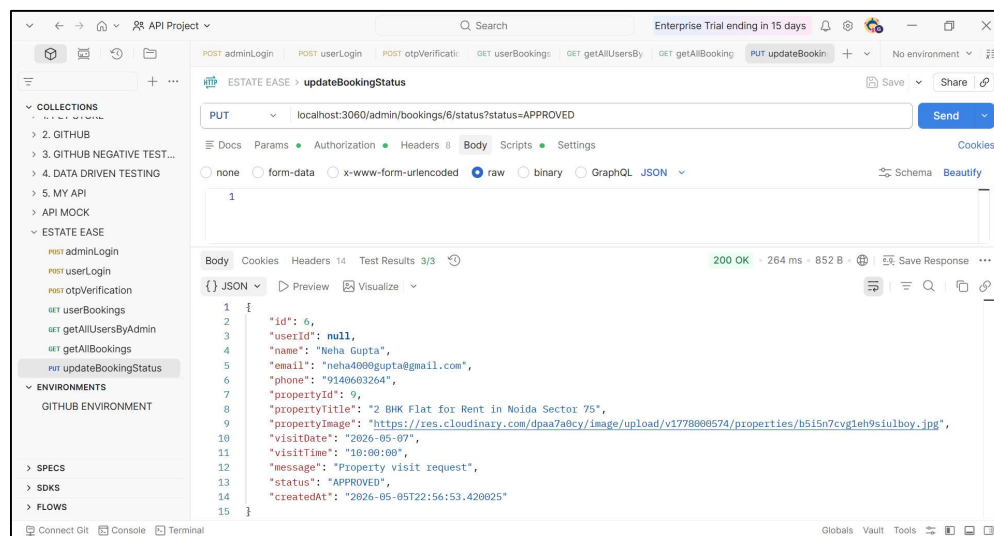


Fig 21: updateBookingStatus

Request Details

The API request was sent using the PUT method with the booking ID and updated status included in the endpoint URL. The request was authenticated using Bearer Token authorization.

Response Validation

The API returned status code **200 OK** and successfully updated the booking request status in the database. The response contained updated booking information including customer details, property details, visit date, and booking status.

Sample Response Data

```
{
  "id": 6,
  "name": "Neha Gupta",
  "email": "neha4000gupta@gmail.com",
  "propertyTitle": "2 BHK Flat for Rent in Noida Sector 75",
  "visitDate": "2026-05-07"
}
```


4.5.8 DELETE USER API TESTING

The Delete User API was tested using Postman to verify successful deletion of registered users by the administrator. The API allows the admin to remove user accounts from the Estate Ease application database.

API Endpoint	HTTP Method	Purpose
/admin/users/{id}	DELETE	Delete registered user

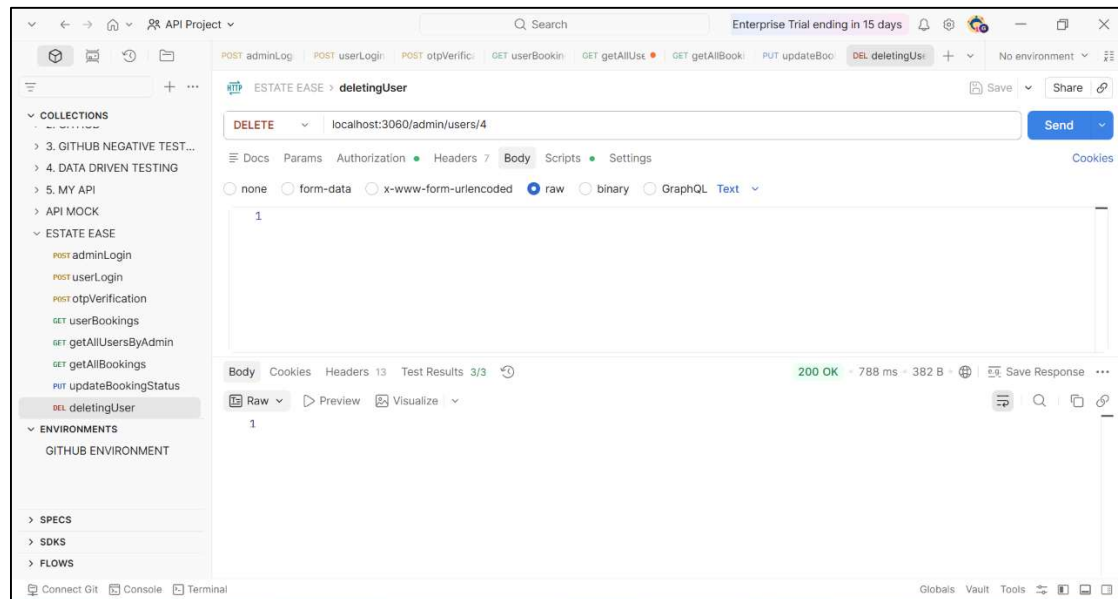


Fig 22: deletingUser

Request Details

The API request was sent using the DELETE method with the user ID included in the endpoint URL. The request was authenticated using Bearer Token authorization to provide secure admin access.

Response Validation

The API returned status code **200 OK** and successfully deleted the selected user from the database. This verified proper execution of the delete operation and successful admin authorization.

4.6 POSTMAN TEST REPORT (APPENDIX A)

A Postman test report was generated after executing the API test collection. The report contains request execution details, response status, execution time, and test validation results for different API endpoints tested during the project.

CHAPTER 5 - SYSTEM IMPLEMENTATION

5.1 SYSTEM IMPLEMENTATION

The Estate Ease application was successfully implemented as a web-based property booking and management system. The frontend was developed using React.js, while the backend was implemented using Java and Spring Boot. MySQL was used for database management.

Different modules such as user authentication, property management, booking management, and admin operations were integrated successfully through REST APIs. The implemented system was tested using Functional Testing, Integration Testing, System Testing, and API Testing using Postman.

5.2 DEPLOYMENT REQUIREMENTS

The application requires a standard hardware and software environment for successful execution. The system was executed in a local development environment using Java, Spring Boot, React.js, MySQL, and Postman.

The minimum hardware requirements include an Intel Core i5 processor, 8 GB RAM, and stable internet connectivity. Visual Studio Code and MySQL Workbench were used during development and testing activities.

5.3 USER ROLES AND ACCESS

The application provides separate access for users and administrators.

Users can register, log in, search properties, schedule visits, submit booking requests, and send enquiries through the application.

Administrators can manage properties, users, bookings, and customer enquiries through the admin dashboard. Role-based access control was implemented to restrict protected operations to authorized administrators only.

5.4 SECURITY AND AUTHENTICATION

Spring Security was used to implement authentication and authorization within the application. OTP verification and token-based authentication mechanisms were implemented to provide secure user access and protect backend APIs.

Bearer Token authorization was also used during API testing to verify secure communication between frontend and backend modules.

5.5 SYSTEM OUTCOME

The Estate Ease application was implemented and tested successfully according to the specified requirements. Different modules and APIs functioned correctly during testing, and smooth communication was established between frontend, backend, and database components.

The project provided practical experience in software testing, API validation, and web application development using modern technologies and testing tools.

SUMMARY & CONCLUSION

The Estate Ease project was successfully developed and tested as a web-based property booking and management system. The application provides functionalities such as property searching, booking requests, visit scheduling, user authentication, and administrative management through a centralized platform.

The major contribution in this project focused on software testing and API validation. Different testing techniques including Functional Testing, Integration Testing, System Testing, and API Testing using Postman were performed to verify the proper functioning of the application modules and backend services.

Various REST APIs related to user authentication, booking management, property management, and administrative operations were tested using different HTTP methods such as GET, POST, PUT, and DELETE. The testing process helped validate request handling, response generation, data transfer, and secure communication between frontend and backend modules.

The testing activities also helped identify and resolve issues related to API responses, booking workflows, authentication handling, and module integration. After successful testing and validation, the application functioned correctly according to the specified project requirements.

Overall, the project provided practical knowledge and hands-on experience in software testing, API validation, system integration, and web application development using modern technologies and testing tools.

REFERENCES

- [1] <https://nodejs.org/docs/latest/api/> [last accessed on 01/03/2026]
- [2] <https://spring.io/projects/spring-boot> [last accessed on 01/03/2026]
- [3] <https://dev.mysql.com/doc/> [last accessed on 15/03/2026]
- [4] <https://dev.mysql.com/doc/refman/9.7/en/> [last accessed on 15/03/2026]
- [5] <https://www.postman.com/> [last accessed on 01/04/2026]
- [6] <https://learning.postman.com/docs/introduction/overview/> [last accessed on 03/04/2026]
- [7] <https://code.visualstudio.com/docs> [last accessed on 10/04/2026]
- [8] <https://www.postman.com/api-platform/api-documentation/> [last accessed on 15/04/2026]

APPENDIX - A

Newman Report

Collection ESTATE EASE
Time Thu May 07 2026 12:45:05 GMT+0530 (India Standard Time)
Exported with Newman v6.2.2

	Total	Failed
Iterations	1	0
Requests	8	0
Prerequisite Scripts	0	0
Test Scripts	8	0
Assertions	24	5

Total run duration 5.3s
Total data received 4.89KB (approx)
Average response time 567ms

Total Failures 5

Requests

adminLogin			
Method	POST		
URL	http://localhost:3060/admin/login		
Mean time per request	161ms		
Mean size per request	418B		
Total passed tests	3		
Total failed tests	0		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	1	0

userLogin			
Method	POST		
URL	http://localhost:3060/api/auth/send-otp		
Mean time per request	4.2s		
Mean size per request	21B		
Total passed tests	2		
Total failed tests	1		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	0	1

otpVerification			
Method	POST		
URL	http://localhost:3060/api/auth/verify-otp		
Mean time per request	20ms		
Mean size per request	22B		
Total passed tests	1		
Total failed tests	2		
Status code	400		
Tests	Name	Pass count	Fail count
	Status code is 200	0	1
	Status code name has string OK	0	1
	Response time is less than 1000ms	1	0

userBookings			
Method	GET		
URL	http://localhost:3060/bookings/user/waffles.neha@gmail.com		
Mean time per request	72ms		
Mean size per request	848B		
Total passed tests	3		
Total failed tests	0		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	1	0

getAllUsersByAdmin			
Method	GET		
URL	http://localhost:3060/admin/users		
Mean time per request	23ms		
Mean size per request	309B		
Total passed tests	3		
Total failed tests	0		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	1	0

getAllBookings			
Method	GET		
URL	http://localhost:3060/admin/bookings		
Mean time per request	35ms		
Mean size per request	2.9KB		
Total passed tests	3		
Total failed tests	0		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	1	0

updateBookingStatus			
Method	PUT		
URL	http://localhost:3060/admin/bookings/2/status?status=REJECTED		
Mean time per request	27ms		
Mean size per request	423B		
Total passed tests	3		
Total failed tests	0		
Status code	200		
Tests	Name	Pass count	Fail count
	Status code is 200	1	0
	Status code name has string OK	1	0
	Response time is less than 1000ms	1	0

deletingUser			
Method	DELETE		
URL	http://localhost:3060/admin/users/3		
Mean time per request	15ms		
Mean size per request	0B		
Total passed tests	1		
Total failed tests	2		
Status code	403		
Tests	Name	Pass count	Fail count
	Status code is 200	0	1
	Status code name has string OK	0	1
	Response time is less than 1000ms	1	0

Failures

AssertionError: Response time is less than 1000ms	
Description	expected 4183 to be below 1000
Location	assertion:2 in test-script
Request	userLogin

AssertionError: Status code is 200	
Description	expected response to have status code 200 but got 400
Location	assertion:0 in test-script
Request	otpVerification

AssertionError: Status code name has string OK	
Description	expected response to have status reason 'OK' but got 'Bad Request'
Location	assertion:1 in test-script
Request	otpVerification

AssertionError: Status code is 200	
Description	expected response to have status code 200 but got 403
Location	assertion:0 in test-script
Request	deletingUser

AssertionError: Status code name has string OK	
Description	expected response to have status reason 'OK' but got 'Forbidden'
Location	assertion:1 in test-script
Request	deletingUser